
Algebra-EBM: Compositional Energy-Based Reasoning for Symbolic Algebra

Matt Krasnow
Harvard University
mkrasnow@college.harvard.edu

Jaray Liu
Harvard University
jarayliu@college.harvard.edu

Abstract

We propose a compositional approach to symbolic algebra reasoning using energy-based models (EBMs). Rather than training a single model on complete multi-step solution sequences, we train separate energy functions for individual algebraic rules, composing them at inference time to solve multi-rule problems. Building on the work of IRED (Iterative Reasoning through Energy Diffusion), we compute rule-level energies and sum them to guide optimization toward valid solution states. On multi-rule problems, our compositional model improves accuracy by 6.2 percent over a monolithic IRED baseline (13.2% vs 7.1%), achieving an 87% relative improvement in reasoning capabilities. We find this advantage also grows with problem complexity, increasing from a +0.7% advantage on 2-rule problems to a +16.0% advantage on 4-rule chains. Our modular design supports runtime constraint injection and provides a practical mechanism for systematic generalization in neural reasoning systems.

1 Introduction

1.1 Problem Statement and Motivation

Neural systems for symbolic reasoning are often trained on complete multi-step solution trajectories, which can conflate performance with the specific distributions of steps seen during training. As a result, models may struggle to solve novel combinations of familiar steps without additional retraining, rendering many existing intelligent systems fragile to out of distribution tasks.

Energy-based models provide an alternative approach towards generalizable reasoning. Rather than learning a single mapping from problem statements to final solutions, an energy-based approach learns separate energy functions encoding individual reasoning operations or constraints. At inference time, these functions can be composed with no additional training to define an objective whose minima correspond to states that satisfy multiple constraints at the same time. This modular composition enables zero-shot generalization to unseen rule combinations, avoiding retraining on the exponentially large space of possible rule sequences.

In our paper, we examine the potential advantages of energy-based approach towards generalizable reasoning in the specific field of algebra. Specifically, we explore the research question: Can a model solve multi-rule problems after training only on single-rule data?

1.2 Contribution

Our work makes three primary contributions to the EBM literature:

- We define and evaluate rule-level energy composition for symbolic algebra reasoning.
- We extend IRED with modular, rule-specific energy functions composed by energy summation at inference time.

- We show improved performance on multi-step algebraic problems while matching single-rule performance.

2 Related Work

Classical energy-based models (EBMs) assign a scalar energy to each configuration and perform inference by searching for low-energy states [LeCun et al., 2006], framing problem solving as search over the trained energy landscape.

IREN (Iterative Reasoning through Energy Diffusion) [Du et al., 2024] extends classical EBMs by formulating reasoning as optimization over annealed energy landscapes. The model learns an energy that scores compatibility of candidate outputs conditioned on the input, then applies gradient-based diffusion steps to reason towards a valid solution state at inference time. IREN reports strong out-of-distribution performance on structured tasks, including Sudoku, path-finding, and matrix completion, by adapting the number of optimization steps at inference time [Du et al., 2024].

Our work extends upon IREN’s optimization-based inference with modifications to the problem modeling granularity. In IREN, reasoning trains a monolithic energy function per task family, which can convolute multiple constraints and rules. In our paper, we instead train separate energies for individual algebraic rules (distribution, combining like terms, isolation, division) and compose them at inference time. This design mirrors classical modular constraint systems, with the caveat that we work with a differentiable, energy-based architecture.

Other work explores compositional structure in constraint solving with diffusion-style methods. Diffusion-CCSP [Yang et al., 2023b] factors continuous constraint satisfaction problems into simpler constraints and composes them at inference. Recurrent Transformer approaches to constraint satisfaction [Yang et al., 2023a] solve CSP instances end-to-end without explicit energies, emphasizing learned constraint satisfaction rather than explicit compositional energy terms. While these methods focus on continuous or general CSP settings, we target discrete symbolic algebra with explicit rule semantics.

In mathematical and symbolic reasoning, energy-based methods remain less common than sequence models. Large-scale studies of neural mathematical reasoning [Saxton et al., 2019] and symbolic mathematics [Lample and Charton, 2019] primarily use sequence-to-sequence or Transformer architectures. These methods often learn monolithic mappings from problems to solutions rather than reusable, rule-specific components. Our paper targets this gap by investigating whether composing interpretable rule-level energies improves multi-step generalization compared to monolithic approaches.

3 Methodology

3.1 Algebraic Formulation and Definition

To train our energy models, we use single-variable linear equation examples of the form $ax + b = cx + d$ using sequences of rule applications. Let $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$ denote the following four rules which we refer to extensively in this paper:

- **Distribution** (r_1): $a(bx + c) \rightarrow abx + ac$
- **Combine** (r_2): $ax + bx \rightarrow (a + b)x$
- **Isolate** (r_3): $ax + b = c \rightarrow ax = c - b$
- **Division** (r_4): $ax = b \rightarrow x = \frac{b}{a}$ (when $a \neq 0$)

We define the state space $\mathcal{X}$ as the set of valid expressions representable in our domain. Therefore, each rule r_i defines a transformation function $f_{r_i} : \mathcal{X} \rightarrow \mathcal{X}$. The process of algebraic solving can then be represented as a sequence of T states $\{e_t\}_{t=0}^T$ where $e_{t+1} = f_{r_{i_{t+1}}}(e_t)$ and the terminal state e_T has the simplified form $x = v$.

Put simply, given an initial equation e_0 , we can apply a series of algebraic rules to reach a final solved form $x = v$ for some $v \in \mathbb{R}$.

3.2 Equation Representation

To allow gradient-based optimization, we must encode discrete algebraic equations into a continuous 128-dimensional vector space. Let us define $\mathcal{E}$ as the space of valid equation strings in our domain.

We first define an encoder $\phi : \mathcal{E} \rightarrow \mathbb{R}^{128}$ which maps equation strings to the continuous embedding space. Specifically, we use a character level encoding scheme, with each character in the equation mapped to a one-hot vector over a universe $\mathcal{U}$ of 24 symbols (digits, operators, parentheses, and the variable x). For a string of maximum length $L = 64$, we flatten the one-hot representation into a vector of dimension $L \times |\mathcal{U}|$ and project it through a learned linear layer:

$$\phi(e) = \frac{W \cdot \text{flatten}(\text{onehot}(e))}{\|W \cdot \text{flatten}(\text{onehot}(e))\|_2} \quad (1)$$

where $W \in \mathbb{R}^{128 \times (L \cdot |\mathcal{U}|)}$ is the projection matrix. We normalize the embeddings to improve stability throughout training and the diffusion based inference procedure.

Moreover, since our downstream inference procedure operates in this continuous embedding space, we must also map the final embedding back to a discrete equation. To solve this, we use a nearest neighbor decoder based on a candidate set $\mathcal{C} \subset \mathcal{E}$ of valid equations with predefined embeddings $\{\phi(c)\}_{c \in \mathcal{C}}$. Thus, we decode an embedding z as follows:

$$\text{decode}(z) = \arg \min_{c \in \mathcal{C}} \|z - \phi(c)\|_2$$

Note that during evaluation, we construct the candidate set from all target equations in the test dataset. This approach avoids the issue of decoding invalid equations since all candidates are verified equations. As such, we guarantee all decoded solutions after inference are syntactically valid equations.

3.3 Rule-based Energy

We then define individual energy functions $E_i(e, s, t) : \mathcal{X} \rightarrow \mathbb{R}$ for each rule $r_i \in \mathcal{R}$, where the energy landscape over candidate outputs is conditioned on the input equation e and diffusion timestep t . In line with classical EBM work, we train each function E_i to assign low energy for states corresponding to valid applications of rule r_i and higher energy to mismatched expressions. Thus, our rule based energy functions measure how compatible an input equation and candidate output are given the specific rule constraint, with low energy corresponding to more "correct" outputs and high energy corresponding to incorrect outputs conditioned on the input e .

Then, for each rule r_i , we construct a dataset $\mathcal{D}_i = \{(e^{(j)}, s^{(j)})\}_{j=1}^{N_i}$ where each example contains an initial expression $e^{(j)}$ and a target solution $s^{(j)}$ obtained by applying only rule r_i . Further, we parameterize E_i with an MLP network and train with an objective function that combines denoising score matching with an explicit contrastive energy loss:

$$\mathcal{L}_i = \mathcal{L}_{\text{score}} + \lambda \mathcal{L}_{\text{contrast}}, \quad (2)$$

$$\mathcal{L}_{\text{score}} = \mathbb{E}_{k, \epsilon} \left[\|\nabla_{\tilde{s}} E_i(e, \tilde{s}_k, t) - \epsilon\|^2 \right], \quad (3)$$

$$\mathcal{L}_{\text{contrast}} = (E_i(e, s, t) - \tau^+)^2 + (E_i(e, s^-, t) - \tau^-)^2 + \max(0, m - E_i(e, s^-, t) + E_i(e, s, t)) \quad (4)$$

where $\tilde{s}_k$ is the target corrupted by Gaussian noise at step k and s^- denotes an incorrect solution (negative solution). Note that the first loss term (denoising score matching) trains the model to predict the noise direction for iterative refinement, while the second term (contrastive) enforces explicit energy gaps between valid and invalid outputs, preventing flat landscapes and preserving output distinguishability.

3.4 Compositional Reasoning

For multi-rule problems, we compose rule energies in a zero shot manner using weighted summation. Let us define $\mathcal{R}_{\text{active}} \subseteq \mathcal{R}$ as the subset of algebraic rules relevant to a given problem. We then define

the compositional energy as follows:

$$E_{\text{comp}}(x; \mathcal{R}_{\text{active}}) = \sum_{r_i \in \mathcal{R}_{\text{active}}} \lambda_i E_i(x), \quad (5)$$

where $\lambda_i \geq 0$ are composition weights. For simplicity, we use uniform weights $\lambda_i = 1$ in our experiments. However, we hope to continue our research beyond CS2821r.

Since each E_i encodes a rule-specific preference, summation naturally combines these preferences into a single landscape that encourages states compatible with multiple rules. The idea is that candidate output states that are closest to the target solution will correspond to low energy valleys across several rule energy landscapes, yielding a combined lower energy than candidate solutions which only apply one or two rules correctly.

3.5 Inference

During inference, we extend the optimization procedure outlined in IRED to use E_{comp} . Concretely, given an initial equation e_0 and active rules $\mathcal{R}_{\text{active}}$, we optimize using the energy landscape defined by E_{comp} according to the following algorithm:

Algorithm 1 Compositional IRED Inference

```

1: Input: Input embedding  $z_{\text{in}}$ , rule energies  $\{E_i\}$ , landscapes  $K$ , steps  $T$ 
2: Output: Solution embedding
3: Initialize  $z \sim \mathcal{N}(0, I)$ 
4: Define  $E_{\text{comp}}(z) = \sum_i E_i(z_{\text{in}}, z)$ 
5: for  $k = 0$  to  $K - 1$  do
6:   for  $t = 0$  to  $T - 1$  do
7:      $z' \leftarrow z - \eta \nabla_z E_{\text{comp}}(z)$ 
8:     Accept  $z \leftarrow z'$  with  $p = \exp(-\Delta E/\tau)$ 
9:   end for
10:  Scale  $z$  for next landscape
11: end for
12: return decode( $z$ )

```

The algorithm iterates through increasingly sharp energy landscapes conditioned on diffusion time steps, at each step performing T optimization steps. Moreover, we use an annealed noise schedule to escape local minima while allowing convergence in later time steps. We hypothesize that combining these disjointly trained rule-based energy landscapes into a composed function can successfully guide optimization through multi-step solutions *without any* prior training on multi-rule sequences.

4 Experimental Setup

4.1 Single-Rule Training Datasets

Prior to training, we generate a separate synthetic dataset $\mathcal{D}_i$ for each of the 4 rules $r_i \in \mathcal{R}$ described in 3.1. Each dataset $\mathcal{D}_i$ contains 50,000 examples of algebraic problems which can be solved only through applying the corresponding rule r_i . Further, we validate each training example using SymPy to guarantee mathematical correctness.

- **Distribution ($\mathcal{D}_1$):** Problems of the form $a(bx + c) = d$, where a, b, c, d are sampled integers from $[-10, 10] \setminus \{0\}$.
- **Combining like terms ($\mathcal{D}_2$):** Problems of the form $ax + bx + c = d$, with coefficients sampled from the same range $[-10, 10] \setminus \{0\}$.
- **Isolation ($\mathcal{D}_3$):** Problems of the form $ax + b = c$ where isolation yields a division-ready equation.
- **Division ($\mathcal{D}_4$):** Problems taking the form of $ax = b$.

Note that each example includes the initial equation and the ground-truth label $x = v$ verified with SymPy. We also enforce unique rational solutions to avoid ambiguous targets.

4.2 Multi-Rule Evaluation Dataset

Similarly, we construct our evaluation set containing 175 problems which individually require between 2 to 4 sequential rule operations:

- **2-rule problems (75 instances):** Two-rule combinations, such as distribution followed by isolation or combining like terms followed by division.
- **3-rule problems (60 instances):** Three-step sequences such as distribution, combining like terms, isolation.
- **4-rule problems (40 instances):** Problems which require the application of all four rules.

Specifically, we generate multi-rule problems by starting from the terminal solved form of $x = v$, with $v \in \mathcal{R}$, and apply inverse transformations to produce a complex initial equation. Through this synthetic generation procedure, we guarantee unique and verifiable solutions to test on. We provide a reference example of a 4-rule problem below:

Example.

$$\begin{aligned}
 2(3x + 4) + 5x = 19 & \xrightarrow{\text{distribute}} 6x + 8 + 5x = 19 \\
 & \xrightarrow{\text{combine}} 11x + 8 = 19 \\
 & \xrightarrow{\text{isolate}} 11x = 11 \\
 & \xrightarrow{\text{divide}} x = 1
 \end{aligned}$$

4.3 Architecture

We implement the Encoder and Decoder as described in 3.2, using an encoder vocabulary size of 24 characters, max sequence length of 64 tokens, and representation dimension of 128. Table 1 summarizes encoder and decoder specifications in more detail.

Table 1: Encoder and Decoder Hyperparameters

Component	Specification
<i>Encoder</i>	
Vocabulary size	24 characters
Max sequence length	64 tokens
Input dimension	$64 \times 24 = 1536$
Output dimension	128
Projection	Linear with orthogonal init (gain = 0.1)
Normalization	L2 unit norm
<i>Decoder</i>	
Method	Nearest neighbor lookup
Distance metric	L2 (Euclidean)
Candidate set	Target equations from dataset

Our energy models use a small MLP architecture which takes the concatenated embeddings of an input equation and a candidate output equation as input, conditioned on a diffusion timestep. Table 2 contains more specifics on the network parameters. Note that each rule uses the same architecture for consistency.

Table 2: Energy Network Architecture (shared across rules)

Component	Specification
Input dimension	256
Hidden dimension	512
Hidden layers	3 (FC $\rightarrow$ FiLM $\rightarrow$ Swish)
Output dimension	128
Time Embedding	Sinusoidal (dim = 128) + 2-layer MLP
Parameters per rule	$\sim 1.3\text{M}$

4.4 Training

We train each rule-specific energy model independently using the paradigm described in 3.3. Note that due to resource constraints, we were only able to train each architecture for 20,000 steps. Table 3 lists the specific training hyperparameters.

Table 3: Training hyperparameters.

Parameter	Value
<i>Optimization</i>	
Optimizer	Adam
Learning rate	10^{-4}
Batch size	2048
Training steps	20,000 per rule
<i>IREN Config</i>	
Number of landscapes (K)	10
Noise schedule	Cosine
Contrastive loss targets	Positive: 1.0, Negative: 15.0
Contrastive margin	10.0
<i>Dataset</i>	
Problems per rule	50,000
Coefficient range	$[-10, 10] \setminus \{0\}$

4.5 Metrics and Evaluation

We monitor model performance using three metrics:

- **Symbolic Accuracy (Main):** Using SymPy, we solve for x in both the predicted and target equations, checking if the solutions match.
- **Invalid Equation Rate:** Fraction of decoded outputs that fail SymPy parsing. This measures how often the nearest neighbor decoding returns syntactically incorrect outputs.
- **Per-Rule Breakdown:** For multi-rule problems, we report accuracy stratified by the number of rules required (2-rule, 3-rule, 4-rule), allowing analysis of how performance degrades with problem complexity.

We compare our compositional rule-based approach against a baseline monolithic model trained on a dataset of complete multi-step algebraic sequences. Importantly, we use the same architecture as the individual energy models to preserve consistency for valid comparison.

In our experiment, we train each rule-specific models on their respective single-rule dataset and evaluate on both single-rule and multi-rule data. We select hyperparameters using a grid search on a small validation set (10% of training data) and track held out test performance.

5 Results

5.1 Single and Multi-Rule Accuracy

We find that compositional energy functions actually improve multi-rule reasoning capability while maintaining comparable single-rule performance compared to the monolithic energy based approach. Monolithic IRED and the compositional model match on single-rule problems (about 29.6% accuracy). On multi-rule problems, our compositional approach improves accuracy by 6.2 percent (13.2% vs 7.1%), an 87% relative increase from the monolithic baseline. Table 4 summarizes the core results.

Table 4: Performance comparison on algebraic equation solving.

Model	Single-Rule Acc	Multi-Rule Acc
Monolithic IRED	29.6%	7.1%
Compositional (Ours)	29.6%*	13.2%
<i>Improvement</i>	<i>+0.0%</i>	<i>+6.2%</i>

Note that compositional results used rule-specific models for single-rule evaluation. To provide a more fine grained comparison, we also test the performance of the compositional and monolithic models on varying levels of problem complexity. We find that the compositional reasoning advantage grows with problem length. On 2-rule problems, the compositional approach achieves a +0.7% advantage. On 3-rule problems, the compositional advantage increases to +1.8%, and on 4-rule problems the compositional approach yields a +16.0% increase in performance compared to the monolithic baseline. Table 5 breaks down multi-rule accuracy in more depth.

Table 5: Multi-rule performance breakdown by number of rules.

Problem Complexity	Monolithic	Compositional	Advantage
2-rule (75 problems)	1.2%	1.9%	+0.7%
3-rule (60 problems)	8.0%	9.8%	+1.8%
4-rule (40 problems)	12.0%	28.0%	+16.0%

Surprisingly, the 4-rule setting demonstrates the largest gap in performance despite the monolithic model being trained primarily on these examples and the compositional models performing out of distribution reasoning.

5.2 Error Analysis

In addition to algebraic accuracy metrics, we analyze incorrect outputs to identify limitations in either model’s performances (compositional and monolithic). We find that the monolithic model exhibits two primary failure modes: (1) *rule interference*, occurring when learning one rule degrades performance on another, and (2) *sequence confusion*, occurring when the model applies the right set of rules in an incorrect order.

By contrast, the compositional model shows more stable per-rule behavior but often fails in two areas: (1) *decoding errors*, occurring when continuous equation embeddings decode to invalid expressions, and (2) *energy balancing*, occurring when one rule energy dominates the others and pushes optimization trajectory toward an incorrect solution valley.

However, we find that both models produce valid syntax in more than 95% of cases. Thus, we suspect representation and decoding quality is not the main driver of errors. Instead, the primary challenge is learning energy landscapes that lead to reliable and robust multi-step optimization processes.

6 Discussion

6.1 Result Analysis

We interpret our results as strong evidence that a compositional approach towards multi-step reasoning is favorable compared to current monolithic approaches, especially given our limited training. We provide three explanations for the observed gap between monolithic and compositional energy-based reasoning on algebra problems. First, each individual rule-based model internalizes a clean representation of its specific rule and therefore reduces cross-rule confusion which arises when training on all rules simultaneously.

Further, our compositional algebra model reuses learned rule-energies in new combinations at test time. Therefore, the model is able to represent multiple subsets of $\mathcal{R}$ through energy summation without a need for training on each multi-rule sequence, promoting a more robust system of generalization by construction. On the other hand, our monolithic model must represent all 4 algebraic rules in one set of parameters, increasing the risk of overfitting on certain rule sequences and internal knowledge entanglement.

Finally, individual rule energies form simpler landscapes around valid local transformations as a product of the simpler task at hand. Summation can then accurately capture multiple rules while maintaining strong performance on single rule tasks. By contrast, a monolithic training objective inherently changes the energy landscape for all other rules while training on another, leading to more complex and convoluted energy landscapes.

Beyond the domain of algebra, our findings suggest broader implications for energy-based approaches to generalizable reasoning. Since energy functions encode compositional constraints over states, they provide a unique approach towards many problem solving tasks. Moreover, their strong zero-shot generalization make them a promising avenue for the development of generally intelligent systems through reusable knowledge and scalable generalization.

6.2 Limitations and Future Work

Our approach faces two main limitations. First, decoding from continuous embeddings to discrete expressions introduces approximation noise into reasoning, especially as equations grow in complexity. Second, our algebra based approach assumes the existence of a well defined rule set while many real world settings involve evolving or unknown rules which present a separate set of challenges for future energy-based reasoning work.

Future work should focus on extending compositional energy-based reasoning into more complex symbolic settings, particularly where the rule set $\mathcal{R}$ is not well defined such as medical diagnosis. Another path for future work could be integrating energy models into the classical AI systems which we have seen in CS 2821r, such as replacing the demons in Selfridge [1959] with EBM. With a learned weighting of the demon energy functions, one could replace the decision demon with a similar compositional reasoning approach.

7 Conclusion

In Algebra-EBM, we introduce compositional energy-based reasoning for symbolic algebra by training rule-level energy functions and composing them at inference time. The approach improves multi-rule accuracy by 6.2% over a monolithic IRED baseline (13.2% vs 7.1%) while matching equivalent single-rule performance.

Our experiments show that the compositional advantage grows with problem length, achieving more than doubled accuracy on 4-rule chains (28.0% vs 12.0%). The method supports modularity, enables constraint injection, and provides a concrete framework for studying systematic generalization through architectural decomposition.

References

- Yilun Du, Jiayuan Li, Igor Mordatch, and Joshua B Tenenbaum. Iterative reasoning through energy diffusion. In *Proceedings of the 41st International Conference on Machine Learning*, pages 12186–12220. PMLR, 2024.
- Guillaume Lample and François Charton. Deep learning for symbolic mathematics. *arXiv preprint arXiv:1912.01412*, 2019.
- Yann LeCun, Sumit Chopra, Raia Hadsell, Marc’Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. *arXiv preprint arXiv:1904.01557*, 2019.
- Oliver G. Selfridge. Pandemonium: A paradigm for learning. In *Proceedings of the Symposium on Mechanization of Thought Processes*, volume 1, pages 511–529. HMSO, 1959.
- Zhaoyu Yang, Dileep George, Sergey Levine, and Pieter Abbeel. Neural constraint satisfaction: Hierarchical abstraction for combinatorial generalization in robot learning. *arXiv preprint arXiv:2305.19582*, 2023a.
- Zhaoyu Yang, Rao Kashyap, Dileep George, Pieter Abbeel, and Sergey Levine. Compositional diffusion-based continuous constraint solvers. In *Conference on Robot Learning*, pages 1249–1259. PMLR, 2023b.